第5章 第1节 为什么需要RAG

- 来源: https://class.imooc.com/lesson/5271#mid=104913
- 阶段: 阶段二: 【实战入门】大模型开发与知识接入
- 周: 第5周 RAG：知识接入技术
- 模块: RAG核心原理
- 课节: 图文 1-1 为什么需要RAG

1-1 为什么需要RAG

第5章 第1节 为什么需要RAG

> [!TIP] 阅读指南 想象一下，正在和一位博学多才的朋友聊天。他知识渊博、谈吐风趣，几乎无所不知。但当问起公司上个月刚发布的新产品细节时，他却支支吾吾，开始"一本正经地胡说八道"——用看似专业的术语拼凑出一些似是而非的答案。 这，就是大模型面临的真实困境。 ChatGPT、通义千问这些大模型确实强大，它们在训练时"读过"整个互联网的内容，掌握了人类知识的精华。但这份知识，却像一本在2023年某个时间点就被封存的百科全书——之后发生的一切，它都不知道；公司内部的文档、客户的私密数据、昨天的新闻，它更是一概不知。 该如何让AI"知道"这些它不知道的信息?

1.1 大模型的两大知识局限

训练数据的时间截止

问题场景

你问DeepSeek：“2025年诺贝尔文学奖得主是谁?”

它可能回答： “我的知识截止日期是2024年7月，因此对于2025年的事件（包括诺贝尔奖）我无法提供真实准确的信息。”

你可能会觉得这个例子和现在大模型的现状不符——“我刚才问DeepSeek这个问题，它能通过联网搜索给我答案啊！”

没错。现在很多AI产品（ChatGPT、DeepSeek、Qwen等）都集成了联网搜索、实时查询等能力。但这些能力本质上就是在使用"检索增强"技术（另一种RAG）。大模型本身无法回答训练数据截止日期之

后的问题。

所以，我们这里讨论的是纯粹的大模型本身。如果剥离掉所有外部增强手段（联网、工具调用、知识库），只让模型凭借训练时"记住"的知识来回答，它就会受限于训练数据的时间截止点。

为什么会这样

我们在第一章里讨论过这个问题：大模型的训练是一个漫长而昂贵的过程。GPT-4的训练可能耗资上亿美元，训练周期长达数月。训练完成后，模型的知识就被"冻结"在那个时间点。除非重新训练，否则它永远活在那个时间截面里，所以它不知道今天的新闻、昨天发布的产品、刚刚更新的政策，也无法获取天气、股价、航班等实时动态信息。

两个问题的本质都是:模型的知识被"冻结"在训练时间点，无法获取之后或当下的信息。

无法访问私有数据

你的公司有几百份内部文档:产品手册、技术规范、FAQ、会议纪要。新员工入职时，HR需要反复回答同样的问题:“年假怎么申请?” “报销流程是什么?” “Wi-Fi密码是多少?”

你想:能不能让AI来回答这些问题?

但当你把问题丢给ChatGPT时，它一脸茫然:“我没有贵公司的内部资料，无法回答。”

大模型的训练数据来自公开的互联网内容。你公司的内部文档、客户的隐私数据、企业的核心资料，这些都不在它的训练语料里。

除非:

- 把这些文档全部上传到公网（隐私泄露风险）
- 用这些数据重新训练一个模型（成本高昂）

否则，AI永远不知道这些信息。

1.2 传统解决方案的困境

面对这些问题，我们曾经尝试过一些传统方案，但它们都有各自的局限。

Fine-tuning

什么是Fine-tuning

简单说，就是拿一个预训练好的大模型，用私有数据再训练一遍，让它"学会"领域知识。但它有一些问题:

成本极高。 需要大量高质量的训练数据（几千到几万条），需要GPU集群进行训练（费用从几千到几万美元），还需要专业的机器学习团队来操作。

更新困难。 每次数据更新都要重新训练——想想刚训练好的模型，又出现了新的数据需要处理。训练周期通常要几天到几周，无法快速响应变化。

灵活性差。 训练好的模型很难修改，发现错误就得重新训练，无法针对单个问题快速调整。

就像让厨师学会做一道新菜，送他去厨艺学校学习三个月。当然能学会，但成本太高、周期太长，而且学完后如果菜谱有调整，还得再学一遍。那有没有更简单的方法？比如直接给他一张菜谱，让他照着做？

Long Context

什么是Long Context

现在的大模型支持越来越长的上下文窗口：

- Gemini 3 Pro：100万tokens（约75万字）
- Qwen3-Max：100万tokens（约75万字）

所以有人想：既然上下文这么长，我直接把所有文档都塞进Prompt不就行了？但这个方法有一些问题：

成本飙升。 Token费用按输入长度计费，每次对话都要传输全部文档，成本是普通对话的几十倍甚至上百倍。

检索困难。 模型需要在海量文本中找信息，准确率随文本长度下降，响应时间也变长。

检索困难。

不可扩展。 文档超过上下文窗口就无法处理，而企业可能有几GB甚至几TB的文档，物理上根本塞不下。

就像去图书馆借书，馆员没有索引系统，而是把整个图书馆的书都搬到你面前，让自己慢慢找。理论上可行，但效率极低，而且搬书的成本谁来承担？

1.3 RAG：优雅的解决方案

RAG（Retrieval-Augmented Generation，检索增强生成）提供了一个优雅而实用的解决方案。

RAG的核心思想

不是让AI"记住"所有知识，而是教会它"查资料"的能力。

工作流程

把RAG想象成：先找对资料，再让AI在这些资料的基础上回答问题，整个过程大致分为三步:

- 用户先提问题，说明自己想了解什么。
- 系统根据问题，在知识库/资料库中检索出几段最有可能相关的内容。
- 把这些内容连同原始问题一起交给AI,由AI综合理解后生成回答。

```
用户提问
   |
   v
检索相关文档（从知识库中找到最相关的3-5段内容）
   |
   v
拼接上下文（把检索到的内容和问题一起给AI）
   |
   v
AI生成回答（基于检索到的真实资料回答）
```

RAG的三大优势

低成本。 不需要重新训练模型，只在需要时检索和传输相关内容，Token消耗远低于Long Context。

案例：某企业有1000份产品文档，总计500万字。

方案A（Long Context）每次对话传输500万字，费用约$50； 方案B（RAG）检索最相关的3段共3000字，费用约$0.03。成本差距：1600倍。

灵活可控。 文档更新立即生效，无需重新训练，可以精确控制AI能访问哪些信息，还可以追溯答案来源，提升可信度。

案例：产品手册更新了一个参数，从"支持5个用户"改为"支持10个用户"。

Fine-tuning方案需要重新训练； RAG方案只需更新知识库文档，立即生效。

可扩展。 知识库可以无限扩展，不受模型上下文窗口限制，支持海量文档（GB甚至TB级别）。

案例：某法律公司有10万份判例文档，总计10GB。

Long Context方案物理上无法实现（远超上下文窗口）； RAG方案将10万份文档向量化存储，检索时只取最相关的3-5份，完全可行。

1.4 RAG与直接搜索的区别

先解答一个可能的疑问：

> “知识库里已经有所有答案了，直接搜索返回不就行了吗？还需要AI干什么？”

通过实际场景看看：只搜索和**RAG（搜索+AI）**到底有什么区别。

只搜索

企业知识库问答

场景背景

公司有几百份文档：产品手册、FAQ、操作指南、会议纪要。新员工入职时，HR需要反复回答同样的问题：“年假怎么申请？”“报销流程是什么？”“Wi-Fi密码是多少？”

假设知识库里有这样几条FAQ：

> 【文档A：年假天数】 Q: 年假有多少天？ A: 入职1年内：5天；1-3年：7天；3-5年：10天；5年以上：15天 【文档B：申请流程】 Q: 年假怎么申请？ A: 登录OA系统 → 选择"请假申请" → 填写年假类型和时间 → 提交给直属领导审批 【文档C：申请限制】 Q: 年假申请有什么限制？ A: 需要提前3天申请；春节、国庆等节假日需提前1个月

员工提问

“我入职2年了，下周想请5天年假去旅游，来得及吗？”

传统方式

系统返回：

> 找到3条相关文档： 【年假天数】 入职1年内：5天；1-3年：7天；3-5年：10天。... 【申请流程】 登录OA系统 → 选择"请假申请"... 【申请限制】 需要提前3天申请；春节、国庆等节假日需提前1个月

员工体验 需要自己读完3段文档，自己计算年假天数，自己判断是否来得及。最关键的问题"来得及吗"没有被直接回答。

问题本质 用户需要的是答案，而不是材料。

问题本质

步骤1：检索知识库（和传统方式一样，找到相同的3段文档）

步骤2：AI理解问题并整合信息

AI看到的完整Prompt：

> 参考资料： 【文档A、B、C的完整内容】 用户问题： 我入职2年了，下周想请5天年假去旅游，来得及吗？ 请基于参考资料回答用户问题。

步骤3：AI生成的回答

> 可以的，来得及！ 根据公司规定： 您入职2年，有7天年假额度，请5天完全够用 普通时段只需提前3天申请，下周去旅游符合要求 建议您现在就登录OA系统提交申请： 选择"请假申请" 填写年假类型和日期（5天） 提交给直属领导审批 祝您旅途愉快！

AI的价值

- 信息整合：从3段文档中提取关键信息
- 逻辑推理：自动计算"入职2年→7天额度→5天够用"
- 个性化：针对"2年" “5天” "下周"这些具体情况定制回答
- 直接回答：明确告诉用户"来得及"

智能客服

电商平台每天收到上万条客户咨询：退货流程、物流查询、产品参数。

假设知识库里有这样的内容：

> 【文档A：退货政策】 7天无理由退货，商品需保持原包装，不影响二次销售。 【文档B：退货流程】 进入订单详情 → 点击"申请退货" → 选择退货原因 → 提交申请 → 等待商家审核 【文档C：退款时间】 审核通过后，退款将在3-5个工作日内退回原支付账户

客户提问

“我昨天买的衣服尺码不合适，现在退货能退多少钱？多久到账？”

传统客服方式

人工客服需要查阅订单、手动查找退货政策、逐条回答客户问题，人力成本高，响应时间长。

固定话术机器人只会复制粘贴文档：

机器人回复：

```
您好，以下是退货相关信息：
1. 退货政策：7天无理由退货...
2. 退货流程：进入订单详情...
3. 退款时间：审核通过后3-5个工作日...

请问还有其他问题吗？
```

客户体验 机器人只会复制粘贴文档，没有回答"能退多少钱"，体验僵硬，像在读使用手册。

步骤1：检索知识库（找到退货相关的3段文档）

步骤2：查询订单信息（通过API获取：订单金额198元，购买时间昨天）

步骤3：AI整合信息并回答

```
您好！您购买的商品（198元）符合7天无理由退货政策，可以全额退款。

退货步骤：
1. 在订单详情页点击"申请退货"
2. 选择"尺码不合适"作为退货原因
3. 提交后等待商家审核（通常1个工作日）
4. 审核通过后，198元将在3-5个工作日内退回您的支付宝账户

温馨提示：请保持商品原包装，不要影响二次销售哦！

需要我帮您直接发起退货申请吗？
```

客户体验满意度大幅提升。

- 跨系统整合：结合知识库（政策）+ 订单系统（金额）
- 个性化回答：针对"昨天买的""198元"定制答案
- 主动服务：不只回答问题，还提供下一步操作建议

核心总结

通过以上两个场景，传统搜索与RAG的根本差异：

传统搜索：找资料——返回原始文档片段，需要人工阅读、理解、整合，像图书馆的卡片索引。

RAG：给答案——理解问题意图，跨文档整合信息，生成针对性的解决方案，像知识渊博的助手。

一句话总结

没有AI，RAG只是个搜索引擎；有了AI，RAG才是智能助手。

1.5 你可能的疑问

Q1：RAG会不会也产生幻觉?

会，但大幅降低。因为AI的回答基于检索到的真实文档，而不是凭空编造。可以要求AI引用来源，验证答案的准确性。

Q2：RAG适合所有场景吗?

不是。如果需要AI深度理解某个垂直领域（比如医疗、法律），Fine-tuning可能更合适。RAG适合90%的企业通用场景。

Q3：RAG的技术门槛高吗?

中等。需要了解向量化、向量数据库、检索策略，但不需要深厚的机器学习背景，会在后续章节逐步介绍具体实现。

Q4：RAG的成本真的很低吗?

是的。检索部分几乎免费（本地计算），生成部分只需传输少量相关文档，Token消耗远低于Long Context。

1.6 RAG：用自然语言查询知识的新方式

换一个角度来看看RAG的优势。

传统资料查询

传统上，如果想系统化地管理资料，需要：

步骤1 结构化存储

```
-- 设计数据库表
CREATE TABLE knowledge (
    id INT,
    title VARCHAR(200),
    content TEXT,
    category VARCHAR(50),
    tags VARCHAR(200),
    created_at DATETIME
);
```

步骤2 手工整理

- 每条知识都要打标签、分类
- 维护表结构、分类体系

步骤3 写SQL查询

```
-- 想找"MySQL索引优化"，得这样写
SELECT * FROM knowledge
WHERE category = 'MySQL'
  AND tags LIKE '%索引%'
  AND tags LIKE '%优化%';
```

但传统方式有几个问题：结构化成本高（手工分类、打标签）；需要会写SQL，普通用户有门槛；只能关键词匹配，记不清用的词就找不到；无法理解人类的自然语言——比如"我踩过哪些坑？"这种语义查询。

大模型的能力

现在，大模型有了一个关键能力：理解自然语言。这意味着你不需要写SQL，直接用人话问；AI能理解你的意图（“踩坑”、"最佳实践"这些语义）；还能跨文档整合信息。

但是，大模型不知道你的私有数据，它只会基于公开的训练数据回答。

RAG：自然语言 + 知识库

RAG就是把这两个能力结合起来：

传统 SQL 方式：

```
+-------------------------------------
| 你的知识库（MySQL） -> SQL查询 -> 结果
|                              ^
|                          需要会写SQL
+-------------------------------------
```

RAG 方式：

```
+-------------------------------------
| 你的知识库（文档） -> 向量化 -> 向量数据库
+-------------------------------------
                          |
                          v
+-------------------------------------
| 自然语言问题 -> 向量化 -> 语义搜索
|                                |
|                                v
|                          找到相关文档
|                                |
|                                v
|                          大模型整合 -> 答案
+-------------------------------------
```

> [!TIP] 上图中的"向量化"，是RAG的核心技术，后续会讲解

核心区别

特性	传统SQL	RAG
存储	结构化(MySQL)	原始文档
查询方式	写SQL语句	自然语言对话
匹配方式	关键词精确匹配	语义理解匹配
返回结果	原始记录	AI生成的答案

假如我们想找"之前解决过的线上接口超时问题"。

SQL方式

```
SELECT * FROM knowledge
WHERE (title LIKE '%超时%' OR content LIKE '%超时%')
   OR (tags LIKE '%接口%' AND tags LIKE '%性能%');
-- 问题：可能漏掉用"timeout"这个英文词记录的那次故障
```

RAG方式

```
你："我之前解决过线上接口超时的问题吗？"

AI："是的，找到了3次相关经验：

1. XX电商项目（2023年5月）
   - 问题：下单接口响应时间经常超过5秒
   - 原因：下游服务串行调用过多
   - 解决：改为并行调用并增加缓存
   ..."
```

传统SQL需要结构化存储、写查询语句和关键词匹配，结果往往是"呆板的"、“不人性化的”。而RAG只需上传文档、用自然语言提问，AI通过语义理解就能给出人性化的答案，像是一个能理解需求的助手在解答——这就是RAG的本质：让大模型的自然语言理解能力与个人知识库结合，得到一份令人满意的答案。

1.7 下节预告

理解了"为什么需要RAG"之后，接下来一个自然的问题是：这个系统在技术层面到底是如何运作的？

本节中提到的"向量"是什么？当用户提问时，文本是如何被转换成向量？向量数据库又是如何在海量文档中找到最相关的几段？这些片段又是怎样和大模型拼接在一起，变成一段自然流畅、可追溯来源的回答？

下一节《RAG核心原理》将带读者从宏观概念走向技术细节，拆解Embedding、向量检索、上下文拼接等关键环节，为后面搭建RAG应用打下坚实的技术基础。

1.8 ■ 学点英语

中文	English	音标	说明
检索增强生成	Retrieval-Augmented Generation (RAG)	/rɪˈtriːvl ɔːgˈmentɪd ˌdʒenəˈreɪʃn/	先检索知识库再让AI基于资料生成的问答技术
微调	Fine-tuning	/faɪn ˈtjuːnɪŋ/	用私有数据对预训练模型进行二次训练以注入领域知识
上下文窗口	Context Window	/ˈkɑːntekst ˈwɪndoʊ/	模型单次能处理的最大Token数量，如100万tokens
知识截止日期	Knowledge Cutoff	/ˈnɑːlɪdʒ ˈkʌtˌɔːf/	模型训练数据的最后时间点，之后的信息模型不知道
幻觉	Hallucination	/həˌluːsɪˈneɪʃn/	AI生成看似合理但实际不正确的信息